Universidad Rafael Landívar.

Facultad de Ingeniería.

Licenciatura en Ingeniería en Informática y Sistemas.

Estructura de Datos II

Docente: Ing. José Arturo Monterroso

Manual Técnico

Estudiante: Fabritzio Alejandro López Castillo

Carné: 1507525

Estudiante: Javier André Huertas Bravo

Carné: 1505825

Quetzaltenango, 28 de octubre de 2026

1. Descripción General de la Arquitectura y Responsabilidades

La Plataforma de Gestión de una Red Eléctrica Municipal es una aplicación de consola desarrollada en C# que modela, administra y optimiza la infraestructura de subestaciones y líneas de transmisión de energía eléctrica de un municipio. El sistema ha sido construido íntegramente desde cero, prescindiendo de las colecciones genéricas nativas de .NET para resolver la lógica interna de las cuatro estructuras obligatorias: Tabla Hash, Árbol de Prefijos (Trie), Conjuntos Disjuntos (Union-Find) y Grafo Ponderado.

El código fuente está organizado de manera modular en tres paquetes lógicos dentro del espacio de nombres raíz RedElectricaMunicipal: la capa de dominio (Modelos), la capa de almacenamiento y algoritmos (Estructuras) y la capa controladora de negocio y persistencia (Servicios), consumidas desde la interfaz interactiva en Program.cs.

Estructura Obligatoria	**Archivo Fuente C#**	**Integrante Responsable**
1. Tabla Hash (Encadenamiento + DJB2 + Rehashing Primo)	Estructuras/TablasHashSubestaciones.cs	Fabritzio
2. Árbol Trie (Prefijos Dinámicos + Autocompletado DFS)	Estructuras/TrieSubestaciones.cs	Javier
3. Union-Find (Compresión de Caminos + Unión por Rango)	Estructuras/UnionDeZonas.cs	Fabritzio
4. Grafo (Listas de Adyacencia, BFS, DFS y Dijkstra)	Estructuras/GrafoRedElectrica.cs	Javier

2. Justificación Técnica de las Estructuras de Datos

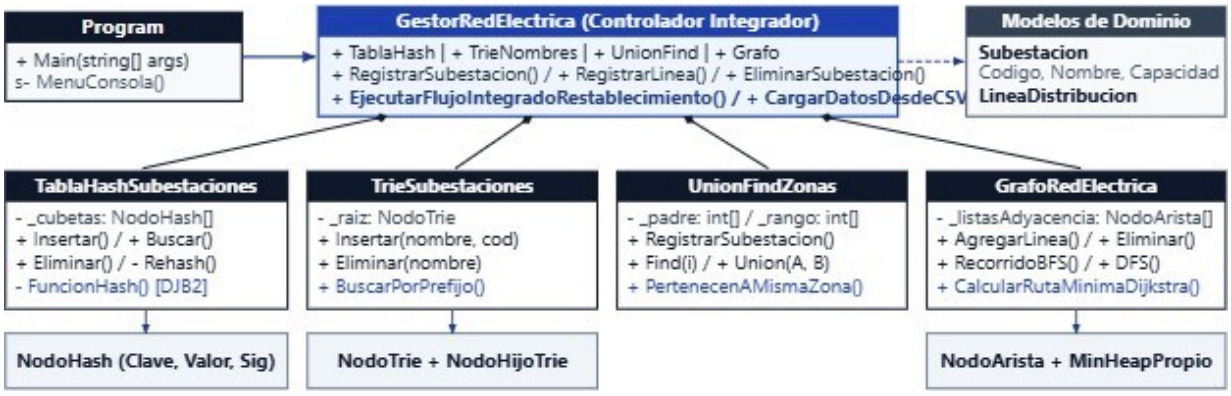
La elección de cada estructura responde directamente a la naturaleza de las operaciones más frecuentes del problema de distribución eléctrica municipal, evitando recorridos secuenciales costosos de orden O(n) sobre el catálogo de subestaciones y líneas:

Estructura	**Operación del Dominio Eléctrico**	**Justificación Técnica de Elección**
Tabla Hash (TablaHashSubestaciones)	Localización directa de fichas técnicas de subestaciones por su código único (ej. SUB-01), inserción y baja de registros.	Permite recuperar la capacidad en MW y el nombre completo de cualquier subestación en tiempo constante promedio **O(1)**. Si se utilizara un arreglo simple o lista enlazada, cada consulta de validación al cargar líneas requeriría O(n), degradando la carga de E líneas a O(E × n) en lugar de O(E).
Árbol Trie (TrieSubestaciones)	Autocompletado interactivo de nombres de subestaciones en tiempo real mientras el operador escribe en consola.	Agrupa los prefijos comunes carácter por carácter en un árbol. Localizar todas las subestaciones que inician con un prefijo de longitud P toma **O(P + K)**, siendo independiente del número total n de subestaciones registradas en el municipio.
Union-Find (UnionFindZonas)	Verificación de pertenencia a la misma zona eléctrica conectada antes de aprobar líneas o ejecutar algoritmos de ruteo.	Determina si dos subestaciones pertenecen a la misma componente conexa en tiempo casi constante **O(1)** gracias a la compresión de caminos y unión por rango. Evita ejecutar el algoritmo de Dijkstra (O((V + E) log V)) cuando dos subestaciones se encuentran en zonas eléctricas aisladas.
Grafo Ponderado (GrafoRedElectrica)	Modelado topológico de líneas bidireccionales y unidireccionales, recorridos de inspección	Las redes de distribución eléctrica son grafos dispersos. El uso de listas de adyacencia propias optimiza el consumo de memoria a **O(V + E)** y, combinado con un MinHeapPropio binario, permite

	(BFS/DFS) y cálculo de ruta de menor costo.	ejecutar el algoritmo de Dijkstra en **O((V + E) log V)**.

3. Diagrama de Clases y Especificación de Componentes

El diseño orientado a objetos concentra la coordinación en la clase GestorRedElectrica, la cual mantiene instancias compuestas de las cuatro estructuras de datos y expone operaciones atómicas a la interfaz de consola.

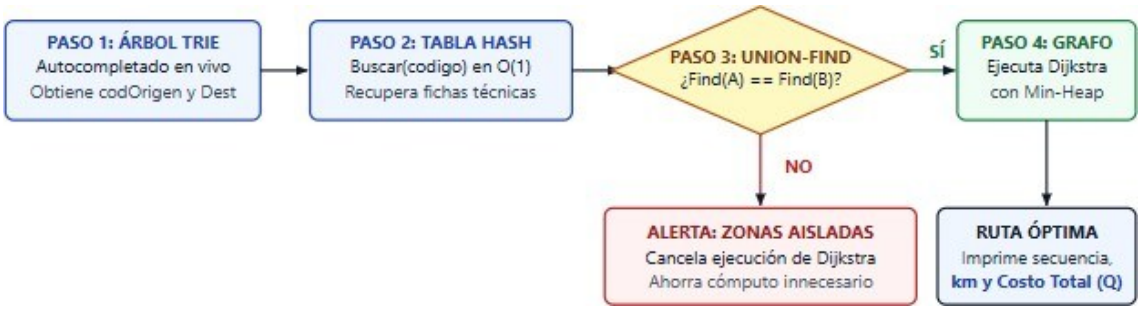


Clase / Nodos	**Atributos Internos Propios**	**Métodos Principales Implementados**
Subestacion LineaDistribucion	Codigo, Nombre, CapacidadMW; CodigoOrigen, CodigoDestino, LongitudKm, CostoMantenimiento, EsUnidireccional.	Constructores con normalización Trim().ToUpper() y sobrecarga de ToString() para formateo técnico.
TablaHashSubestaciones + NodoHash	NodoHash[] _cubetas, int _capacidad, int _cantidadElementos, FACTOR_CARGA_MAXIMO = 0.75.	FuncionHash(clave), Insertar(sub), Buscar(codigo), Eliminar(codigo), Rehash(), ObtenerTodas(), ImprimirEstructuraHash().

TrieSubestaciones + NodoTrie + NodoHijoTrie	NodoTrie _raiz, NodoHijoTrie Hijos, bool EsFinDePalabra, string NombreOriginal, string CodigoAsociado.	Insertar(nombre, codigo), Eliminar(nombre), BuscarPorPrefijo(prefijo, max), RecolectarSugerencias(), ImprimirTodoElTrie().
UnionFindZonas	string[] _codigos, int[] _padre, int[] _rango, int[] _tamanoZona, int _cantidad, int _numZonas.	RegistrarSubestacion(codigo), Find(i), Union(codA, codB), PertenecenAMismaZona(codA, codB), ReiniciarZonas(), ImprimirZonasElectricas().
GrafoRedElectrica + NodoArista + MinHeapPropio	string[] _codigosVertices, NodoArista[] _listasAdyacencia, int[] _vertices, double[] _distancias.	AgregarVertice(cod), AgregarLinea(linea), EliminarLinea(orig, dest), DesconectarVertice(cod), RecorridoBFS(ini), RecorridoDFS(ini), CalcularRutaMinimaDijkstra(), SincronizarUnionFind().
GestorRedElectrica	Referencias públicas de solo lectura a TablaHash, TrieNombres, UnionFind y Grafo.	RegistrarSubestacion(), EliminarSubestacion(), RegistrarLinea(), CargarDatosDesdeCSV(), SeleccionarSubestacionConAutocompletado(), EjecutarFlujoIntegradoRestablecimiento(), GenerarReporteEstrategico().

4. Integración entre Estructuras y Diagrama de Flujo

Para cumplir con el Requisito Técnico #2, ninguna estructura opera de manera aislada. El método EjecutarFlujoIntegradoRestablecimiento() articula las cuatro estructuras en una sola transacción lógica para restablecer el suministro eléctrico al menor costo de mantenimiento entre dos puntos del municipio:



1. **Sincronización en Inserción (RegistrarSubestacion):** Antes de insertar, consulta TablaHash.Buscar(codigo) en O(1); si la subestación ya existía con otro nombre, invoca TrieNombres.Eliminar(existente.Nombre) para evitar claves huérfanas. Seguidamente inserta la entidad en la Tabla Hash, indexa su nombre en el Trie, crea el vértice en el Grafo e inicializa su conjunto disjunto en el Union-Find.
2. **Sincronización en Aprobación de Líneas (RegistrarLinea):** Verifica primero en la **Tabla Hash** que ambas subestaciones existan, consulta UnionFind.PertenecenAMismaZona(origen, destino) para informar si la línea unificará dos zonas separadas o creará un anillo de redundancia, agrega la arista en el Grafo y ejecuta UnionFind.Union(origen, destino).
3. **Sincronización en Eliminación (EliminarSubestacion y EliminarLinea):** Remueve el registro de la Tabla Hash y del Trie, desconecta sus aristas en el Grafo y llama a Grafo.SincronizarUnionFind(UnionFind) para recalcular las zonas eléctricas conectadas.

5. Análisis de Complejidad (Big-O)

A continuación se presenta el análisis asintótico temporal y espacial de cada operación implementada en el proyecto, donde n (o V) representa la cantidad de subestaciones, m la capacidad prima de la Tabla Hash, E la cantidad de líneas de distribución, L la longitud de una cadena, P la longitud del prefijo buscado:

Estructura	Método / Operación	Caso Promedio	Peor Caso	Espacio Auxiliar
Tabla Hash	Insertar(subestacion) Buscar(codigo) Eliminar(codigo) Rehash()	O(1) O(1) O(1) O(n + m)	O(n) (con Rehash) O(n) (colisión total) O(n) (colisión total) O(n + m)	O(n + m) total O(1) por consulta O(1) por baja O(m) arreglo nuevo
Árbol Trie	Insertar(nombre, cod) BuscarPorPrefijo(pref) Eliminar(nombre)	O(L) O(P + K) O(L)	O(L × Σ) O(P + N_{nodos}) O(L × Σ)	O($N_{caracteres}$) O(L_{max}) pila DFS O(1) desmarcado
Union-Find	RegistrarSubestacion() Find(i) (Compresión) Union(codA, codB)	O(n) (busca índice) O(1) O(1)	O(n) O(log n) O(log n)	O(V) arreglos O((n)) recursión O(1) enlaces
Grafo y Min-Heap	AgregarLinea(linea) RecorridoBFS(inicio) RecorridoDFS(inicio) CalcularRutaMinimaDijkstra	O(V + grado(u)) O(V + E) O(V + E) O((V + E) log V)	O(V) O(V + E) O(V + E) O((V + E) log V)	O(V + E) listas O(V) cola BFS O(V) pila DFS O(V + E) en Heap

6. Carga Dinámica de Archivos CSV y Generación de Reportes

El sistema elimina cualquier dependencia de datos quemados en código mediante el método CargarDatosDesdeCSV(rutaSubestaciones, rutaLineas) en GestorRedElectrica.cs. Al iniciar la aplicación (o al invocar la opción correspondiente del menú), el sistema verifica la existencia física de los archivos con File.Exists(), informa la ruta absoluta en caso de error e ignora líneas vacías o comentarios iniciados con #.

Adicionalmente, para respaldar la toma de decisiones, el método GenerarReporteEstrategico ("ReporteRedElectrica.txt") recorre la totalidad de las cubetas de la Tabla Hash mediante ObtenerTodas() en O(n + m), totaliza la capacidad instalada en megavatios (MW) del municipio y exporta un informe ejecutivo fechado a disco.